MODULE 5

COMPLEXITY CLASSES

BIN PACKING

can be implemented by using 2 methods.

1. Online Algorithm : Get Realtime datas.
2. Offline Algorithm : didn't get Realtym datas.

Minimum no.of bin = $\left\lceil \frac{\text{Total weight}}{\cancel{\text{no. of bin}}\ \text{bin capacity}} \right\rceil$

→ Best fit, worst fit, next fit, first fit.

→ First fit decreasing, Best fit Decreasing

Next Fit Algorithm

Q. Apply next fit bin packing approximation algo on the following items with bin capacity = 10. Assuming size of items be $\{5, 7, 5, 2, 4, 2, 5, 1, 6\}$

ans:- Minimum no. of bin $= \left\lceil \frac{\text{total weight}}{\text{bin capacity}} \right\rceil$

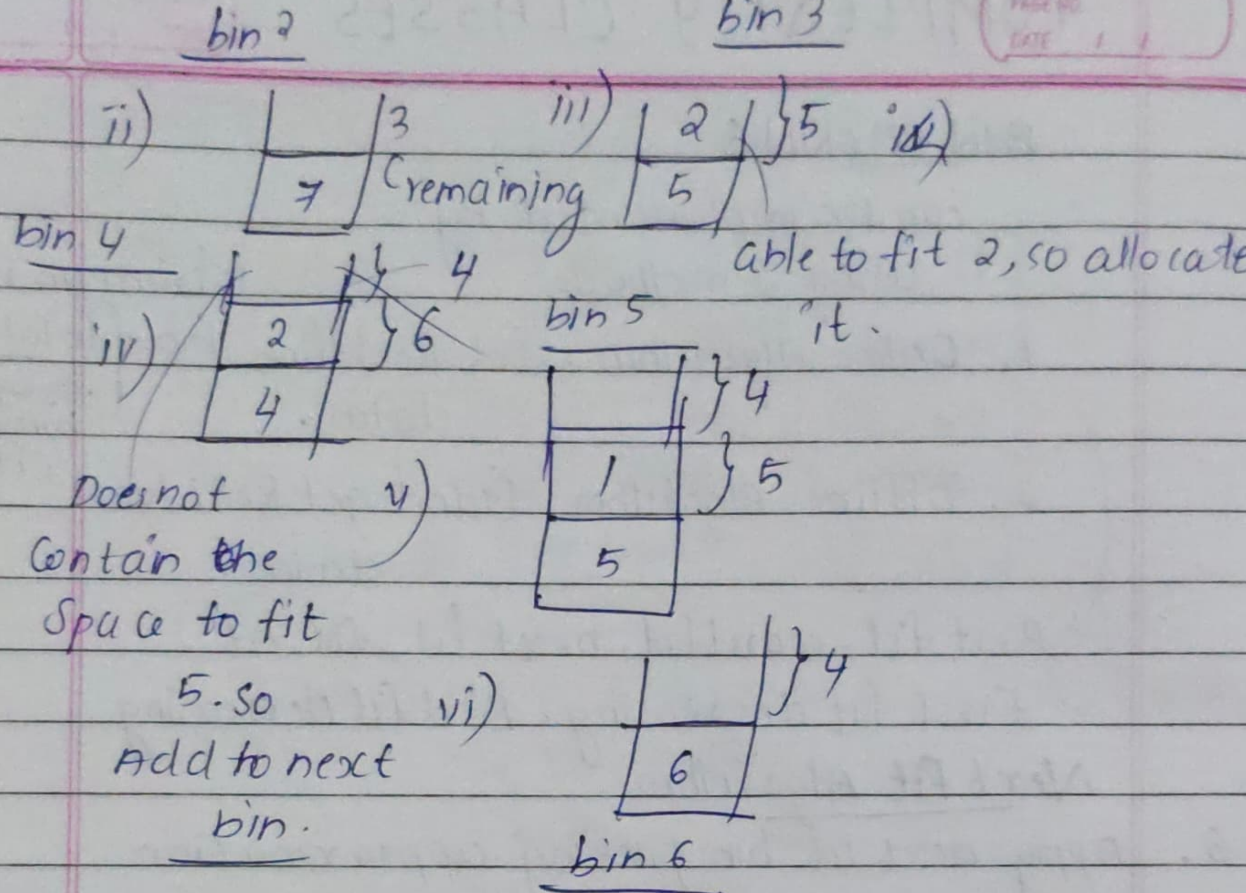
$$= \left\lceil \frac{5+7+5+2+4+2+5+1+6}{10} \right\rceil$$

$$= \left\lceil \frac{37}{10} \right\rceil$$

$$= \lceil 3.7 \rceil = 4$$

i) First Allocate 5 in bin 1 :- [5] } remaining 5.

Total 10

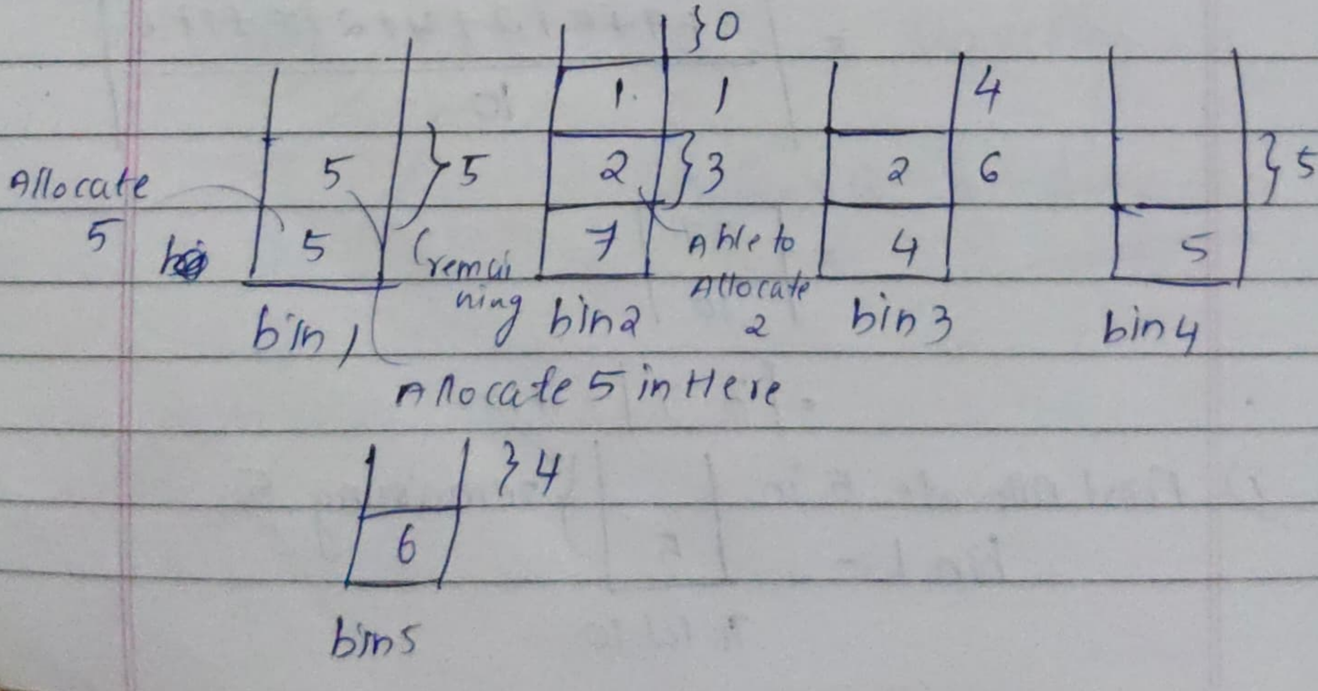


FIRST FIT

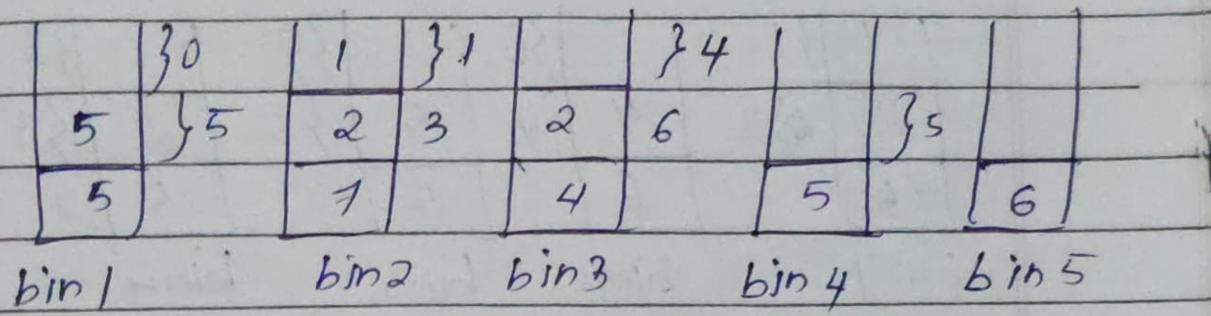
size = {5, 7, 5, 2, 4, 2, 5, 1, 6}

bin capacity = 10

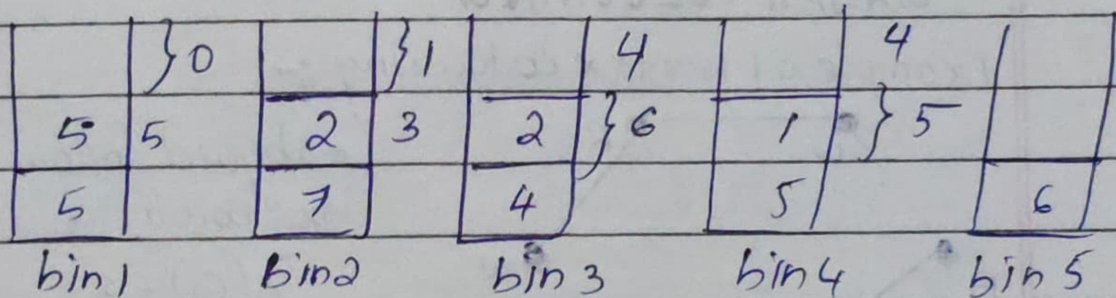
minimum no. of bins = 4.



BEST FIT



WORST FIT



OFFLINE ALGORITHMS

1. Best fit Decreasing
2. first fit Decreasing.

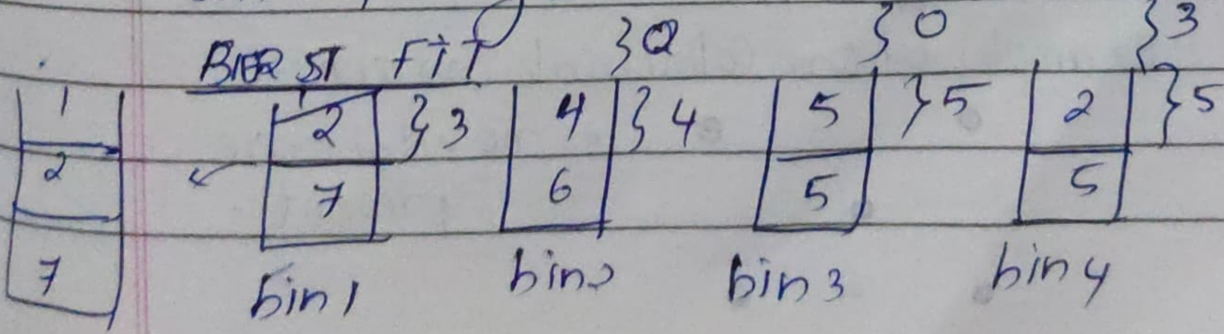
steps :- i) First sort in decreasing order.
ii) then apply BF and FF.

Decreasing order.

size = {7, 6, 5, 5, 5, 4, 2, 2, 1}

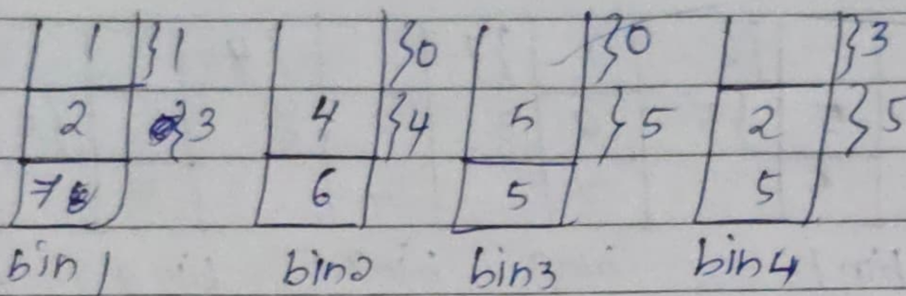
bin capacity = 10

BEST FIT



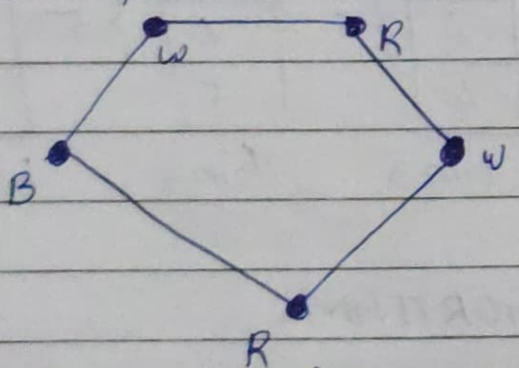
ninte
~~ente~~ ABC-
Thirunavya ~~ninte~~ daddy
- Renita - thiru.

FIRST FIT



GRAPH COLOURING

Example of vertex colouring :-

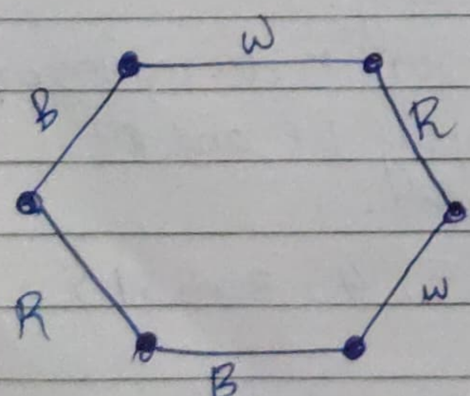


- should follow chromatic no. of colours.

$\lambda(G) = 3$

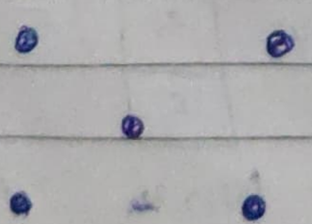
representation of chromatic no.

Example of Edge colouring :-



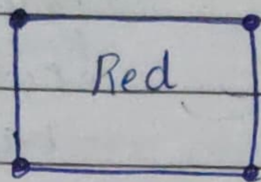
$\lambda(G) = 3$

Example of One Colourable Graph :-



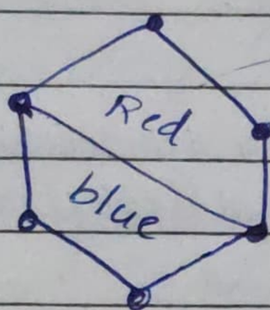
- no edges are present.

Example of face Colouring :-



All should be in 1 Colour.

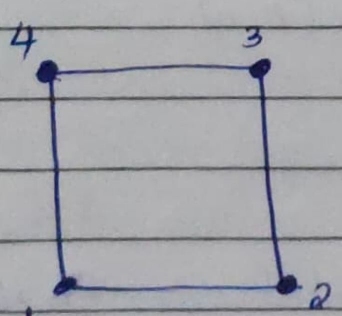
or



partitioning into 2.

```
approximate_graph_colouring (G, n)
{
  for i = 1 to n do
   for c = 1 to n do
     If no Vertex adjacent to vi has color c
   {
     color vi with c
     break
   }
}
```

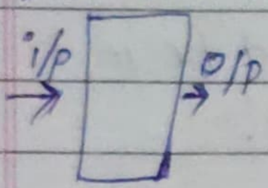
eg:-



C: { W R B G }

RANDOMIZED ALGORITHMS

Deterministic:-

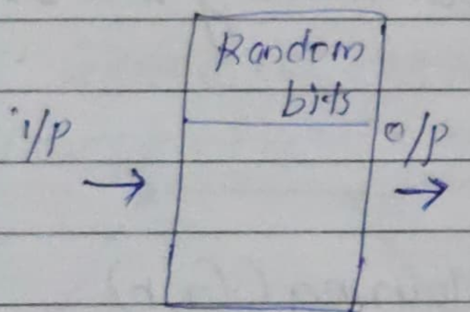


Algorithm

Deterministic Algorithm Running time, Outputs are produced based on the input and its algorithm.

eg:- Linear search, etc.

~~Deterministic:-~~ Randomized:-



Algorithm

Here Random bits are generated and it effects the algorithm.

Eg:- 1. Las vegas Algorithm
2. Monte carlo Algorithm

LAS VEGAS ALGORITHM

Output is always correct and optimal.

Running time is a random no. and also it is not bounded.

eg:- Randomized Quick sort.

Algorithm finding a LV (A, n)

{

Repeat

{

Randomly choose 1 element out of n elements.

}
until ("a is found")
}

RANDOMIZED MONTE CARLO ALGORITHM

- The Algorithm may produce correct output with some probability.
- A monte carlo Algo runs for a fixed no. of steps. ie The running time is deterministic.

Algorithm finding A_MC (A, n, k)
{
i=0
repeat
{
Randomly select 1 element out of n elements
i=i+1
} until (i=k or 'a' is found)
}

RANDOMIZED QUICK SORT

Randomized rand Quick sort (A[ ], low, high)
1. If low >= high, then EXIST
2. while pivot 'x' is not a Central pivot.
   2.1: choose uniformly at random a number

from [low, high]

let the randomly picked number be x.

2.2: Count elements in A [low, high] that are smaller than A [x].

Let this count be sc.

2.3: Count elements is A [low, high] that are greater than A[x]. let this count be gc.

2.4: Let $n = (high - low + 1)$ If $sc >= n/4$ and $gc >= n/4$ then x is a central pivot.

3: partition A [low, high] into subarrays. The first subarrays has all the elements of A that are less than x and the second subarray has all those that are greater than x. Now the index of be pos.

4. randQuicksort (A, low, pos-1)

5. rand Quick sort (A, pos+1, high)

Q) ~~Through~~ prove vertex cover problem is NP Complete.

ans:-

1

step1: polynomial time varification Algorithm

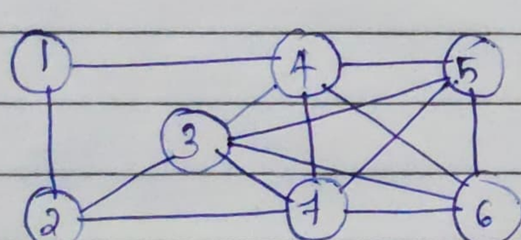
inputs (G, K, V') no. of vertex

Algorithm; step1: Count = 0

step 2: For each vertex v in V' remove all edges adjacent to v from E.

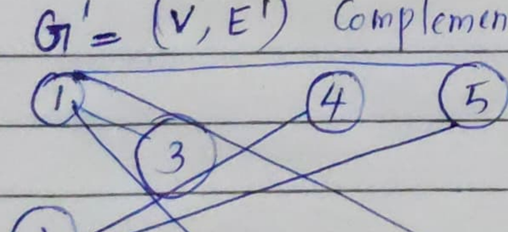
2.1: Increment count by 1
3: If count=k and E is empty then Solution is Correct.
4: Else Solution is incorrect.
Therefore, vertex cover problem belongs to class Np.

II. polynomial time reduction Algorithms
inputs: <G = (V, E), K>
step 1: Consider G' which is complement of G.
step 2: IF G' has a vertex cover of size |V| - K
Then G has a clique of size K.
Therefore, vertex cover belongs to Np Hard.
eg of clique :-



$|V| = 7$
3, 4, 5, 6, 7 $K = 5$
$|V| - K \Rightarrow 7 - 5 = 2$ // size of vertex cover

$G' = (V, E')$ Complement of this graph :-



vertex covers = 1, 2
Here $|V| - K$ = no. of vertex covers in G'. so it is Np hard.

Vertex covers belongs to ~~Verte~~ Np complete.